



DATA STRUCTURES AND ALGORITHMS

23CSE203

Submitted by	
Name	A. Chethan venkata deva
Roll No	AV.SC.U4CSE25008
Year/sem/section	2 nd year/sem-3/CSE-A
Credits	5

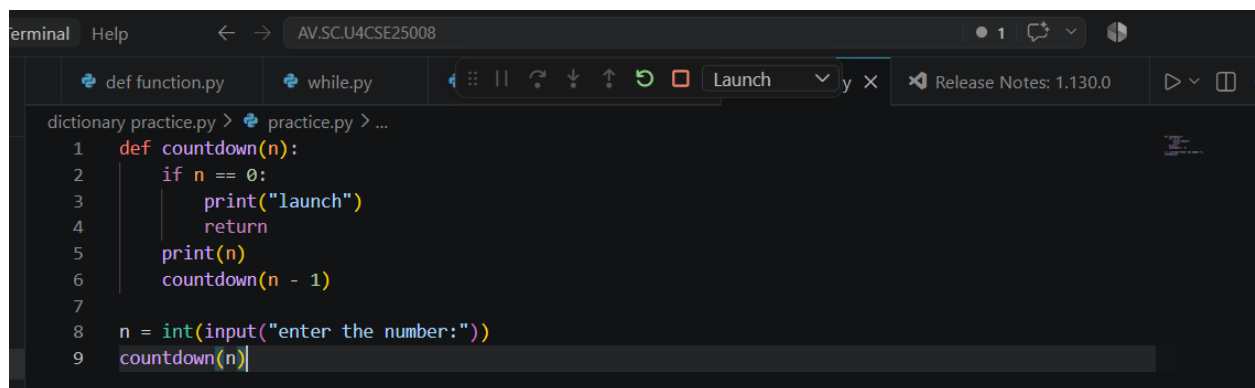
Submitted to	
Name	DR.RAJKUMAR BATCHU
Department	School of computing
Designation	Asst. professor(SI. Grade)

s.no	title	Pageno	remarks
1	WEEK 1: 1a. Recursive countdown from n to Launch. 1b. Recursive calculation of compound growth factor. 1c. Recursive search for employee ID in list. 1d. Recursive factorial calculation for parcel arrangements. 1e. Recursive Fibonacci series generation.		
2	WEEK 2: 2a. Write a python program to perform linear search on the give input. 2b. Write a python program to perform binary search on the given. Verify by taking sorted array as an input. 2c. Write a python program to perform binary search on the given. Verify by taking unsorted array as an input.		
3	WEEK-3: 3a.write a python program to perform bubblesort on the given input 3b. write a python program to perform insertion_sort on the given input 3c. write a python program to perform selection sort by taking given input 3d.write a python program to perform merge sort by taking the given input . 3e. write a python program to perform quick sort by taking given input		

WEEK-1

1A.AIM:A Rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1 followed by “launch”.

Code:

A screenshot of a code editor window. The title bar shows 'terminal', 'Help', and a file path 'AV.SC.U4CSE25008'. Below the title bar, there are tabs for 'def function.py' and 'while.py'. The main editor area shows a Python script for a recursive countdown. The code is as follows:

```
1 def countdown(n):  
2     if n == 0:  
3         print("launch")  
4         return  
5     print(n)  
6     countdown(n - 1)  
7  
8 n = int(input("enter the number:"))  
9 countdown(n)
```

The code is color-coded: 'def' is blue, 'if' is blue, 'n == 0' is blue, 'print' is blue, 'return' is blue, 'print(n)' is blue, 'countdown(n - 1)' is blue, 'n = int' is blue, 'input' is blue, and 'countdown(n)' is blue. The editor also shows a 'Launch' button and a 'Release Notes: 1.130.0' link.

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '62889' '--' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\practice.py'
enter the number:2
2
1
launch
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> 
```

Explanation

Condition 1: Positive Input ($n > 0$)

- The function checks whether $n == 0$.
- Since n is positive, the condition is **False**.
- It executes:
 - `print(n)` → prints the current value of n .
 - `countdown(n-1)` → calls the same function again with $n-1$.
- This process continues until n becomes 0.
- When $n == 0$, the condition becomes **True**.
- It prints **launch**.
- The return statement stops that function call.
- Then the recursive calls return one by one.
- Since there is no value returned by the function, it returns **None**.

Condition 2: Zero Input ($n == 0$)

- The function checks if $n == 0$.
- The condition is immediately **True**.
- It prints **launch**.
- The return statement stops the function.
- No recursive call takes place.
- The function returns **None**.

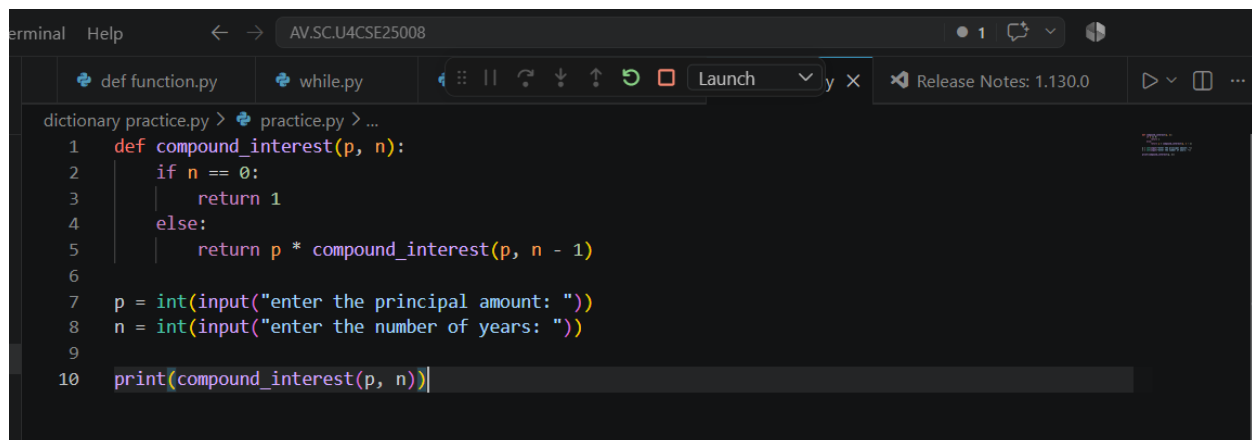
Condition 3: Negative Input ($n < 0$)

- The condition $n == 0$ is **False**.
- It prints the negative value.
- Then it calls itself with $n-1$, making the value even more negative.
- Therefore, it will **never reach 0**.
- The recursion continues indefinitely.
- Eventually, Python raises a **RecursionError** because the maximum recursion depth is exceeded

s.no	Error obtained	Rectification
1	Negative input in countdown	Add a condition to handle negative numbers before recursion.

1B. Aim: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate $p(n)$ where p =principal growth, n = no. of years.

Code:

A screenshot of a code editor window. The title bar shows 'terminal Help' and 'AV.SC.U4CSE25008'. The editor has tabs for 'def function.py', 'while.py', and 'Launch'. The code is in a file named 'practice.py'. It defines a recursive function 'compound_interest(p, n)' which returns 1 if n is 0, and p * compound_interest(p, n-1) otherwise. Below the function, it takes user input for principal amount 'p' and number of years 'n', and prints the result of the function call.

```
dictionary practice.py > practice.py > ...
1  def compound_interest(p, n):
2      if n == 0:
3          return 1
4      else:
5          return p * compound_interest(p, n - 1)
6
7  p = int(input("enter the principal amount: "))
8  n = int(input("enter the number of years: "))
9
10 print(compound_interest(p, n))
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'c:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\Untitled-1.py'
enter the principal amount: 1000
enter the number of years: 1
1000
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation

Condition 1: $n > 0$

- The function `compound_interest(p, n)` is called with the principal amount p and number of years n .
- The function checks whether $n == 0$.
- Since n is greater than 0, the condition is **False**.
- It executes the else block.
- It calculates:
 - $p * \text{compound_interest}(p, n-1)$
- The function calls itself recursively with $n-1$.
- This process continues until n becomes 0.
- Each recursive call multiplies p one more time.
- Therefore, the function calculates p^n .

Condition 2: $n == 0$

- When n becomes 0, the condition $n == 0$ becomes **True**.
- The function returns 1.
- This 1 acts as the **base case** for the multiplication.
- The recursive calls then return one by one.
- Each returned value is multiplied by p .
- Finally, the result is printed using `print(compound_interest(p,n))`

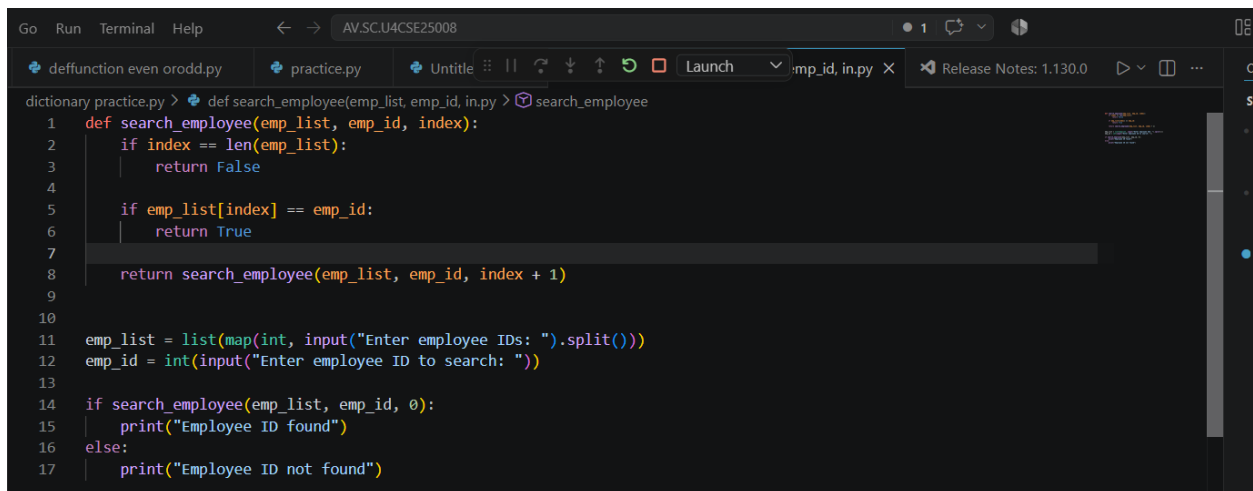
Error table:

s.no	Error obtained	rectification
------	----------------	---------------

1	Wrong indentation	Maintain proper indentation.
---	-------------------	------------------------------

1C. AIM . An Hr department stores employee IDs in a list .Develop a recursive program to search for a given employee ID write a recursive function to search for an employee ID in a list.

Code:

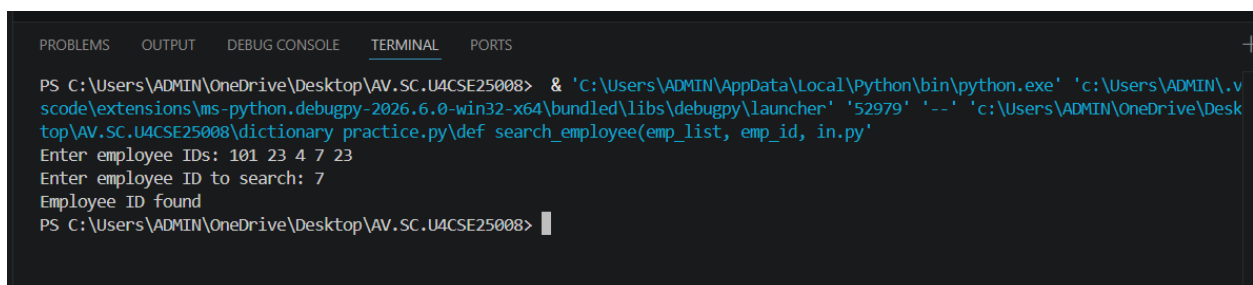


```

Go Run Terminal Help  AV.SC.U4CSE25008  1  Release Notes: 1.130.0
dictionary practice.py > def search_employee(emp_list, emp_id, index):
1  def search_employee(emp_list, emp_id, index):
2      if index == len(emp_list):
3          return False
4
5      if emp_list[index] == emp_id:
6          return True
7
8      return search_employee(emp_list, emp_id, index + 1)
9
10
11 emp_list = list(map(int, input("Enter employee IDs: ").split()))
12 emp_id = int(input("Enter employee ID to search: "))
13
14 if search_employee(emp_list, emp_id, 0):
15     print("Employee ID found")
16 else:
17     print("Employee ID not found")

```

Output:



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'c:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\def search_employee(emp_list, emp_id, in.py'
Enter employee IDs: 101 23 4 7 23
Enter employee ID to search: 7
Employee ID found
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>

```

Explanation

Condition 1: `index == len(emp_list)`

- The function checks whether index has reached the length of the employee list.
- If `index == len(emp_list)`, it means we have checked **all employee IDs**.
- The employee ID was not found.
- The function returns **False**.

- No further recursion takes place.

Condition 2: `emp_list[index] == emp_id`

- The function checks whether the current employee ID is equal to the ID we are searching for.
- If both IDs are equal, the condition becomes **True**.
- The function returns **True**.
- The recursion stops because the employee ID has been found.

Condition 3: Employee ID does not match

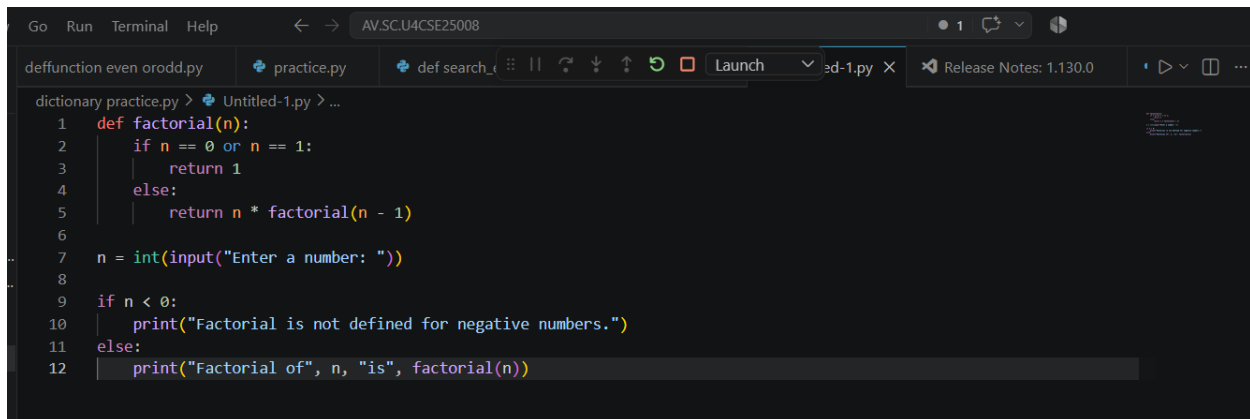
- If the current employee ID is not equal to `emp_id`, the condition is **False**.
- The function calls itself recursively with:
 - `index + 1`
- This moves to the **next employee ID** in the list.
- The process continues until:
 - The employee ID is found, or
 - The entire list is checked.

Error table:

s.no	Error obtained	Rectification
1	Missing return statement	Add the required return statement

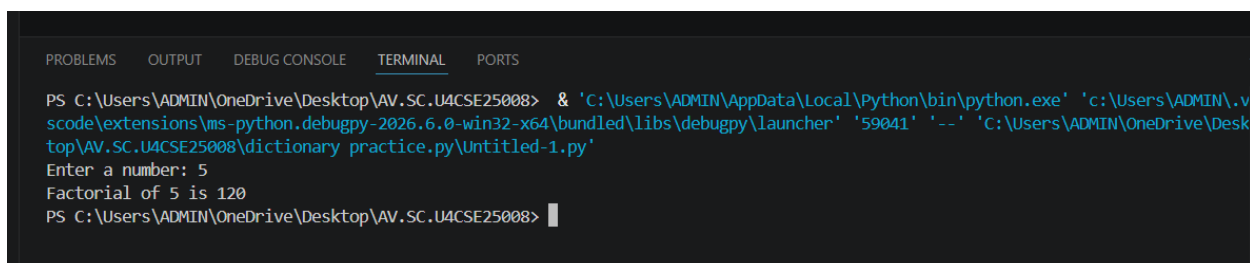
1D. AIM. In a logistics company , the no . of possible ways to arrange in different parcels is calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

Code:

A screenshot of a code editor window. The top bar shows 'Go Run Terminal Help' and a file path 'AV.SC.U4CSE25008'. Below the top bar, there are tabs for 'dictionary practice.py', 'Untitled-1.py', and 'def search_'. The main editor area shows a Python script for calculating factorials. The code is as follows:

```
1 def factorial(n):
2     if n == 0 or n == 1:
3         return 1
4     else:
5         return n * factorial(n - 1)
6
7 n = int(input("Enter a number: "))
8
9 if n < 0:
10     print("Factorial is not defined for negative numbers.")
11 else:
12     print("Factorial of", n, "is", factorial(n))
```

Output:

A screenshot of a terminal window. The top bar shows 'PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS'. The terminal content is as follows:

```
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '59041' '--' 'C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\Untitled-1.py'
Enter a number: 5
Factorial of 5 is 120
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> |
```

Explanation

Condition 1: Negative Input ($n < 0$)

- The program first checks whether $n < 0$.
- If n is negative, the condition is **True**.
- Factorial is not defined for negative numbers.
- It prints:
Factorial is not defined for negative numbers.
- The `factorial()` function is **not called**.

Condition 2: Base Case ($n == 0$ or $n == 1$)

- Inside the `factorial()` function, it checks:
 $n == 0$ or $n == 1$
- If n is 0 or 1, the condition is **True**.
- The function returns **1**.
- This is the **base case** of the recursion.
- It stops further recursive calls.

Condition 3: $n > 1$

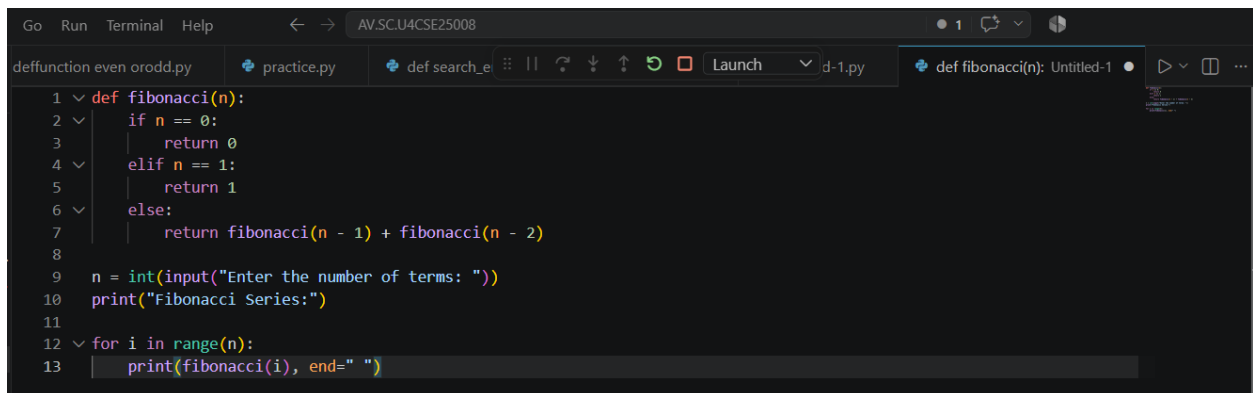
- If n is greater than 1, the condition $n == 0$ or $n == 1$ is **False**.
- The else block is executed.
- It returns:
 $n * \text{factorial}(n - 1)$
- The function calls itself with $n - 1$.
- This continues until n becomes 1.
- Then the returned values are multiplied together

Error table:

s.no	Error obtained	Rectification
1	Wrong recursive value	Updated the recursive value correctly

1E. AIM. Write a recursive program to print the first n terms of the fibonacci series

Code:

A screenshot of a code editor window with a dark theme. The editor shows a Python script for calculating the Fibonacci series. The code is as follows:

```
1 def fibonacci(n):
2     if n == 0:
3         return 0
4     elif n == 1:
5         return 1
6     else:
7         return fibonacci(n - 1) + fibonacci(n - 2)
8
9 n = int(input("Enter the number of terms: "))
10 print("Fibonacci Series:")
11
12 for i in range(n):
13     print(fibonacci(i), end=" ")
```

The editor has multiple tabs open at the top: 'deffunction even or odd.py', 'practice.py', 'def search_e...', and 'def fibonacci(n): Untitled-1'. The 'def fibonacci(n): Untitled-1' tab is currently active. The code is written in a syntax-highlighted style with line numbers on the left.

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '60880' '--' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\Untitled-1.py'
Enter the number of terms: 6
Fibonacci Series:
0 1 1 2 3 5
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> |
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '53127' '--' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\Untitled-1.py'
Enter the number of terms: -8
Fibonacci Series:
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> |
```

Explanation

Condition 1: $n == 0$

- The function checks whether $n == 0$.
- If the condition is **True**, the function returns 0.
- This is the **first base case**.
- No further recursive call takes place.

Condition 2: $n == 1$

- If n is not 0, the function checks whether $n == 1$.
- If the condition is **True**, the function returns 1.
- This is the **second base case**.
- No further recursive call takes place

Condition 3: $n > 1$

- If n is neither 0 nor 1, both conditions are **False**.
- The else block is executed.
- The function calls itself **two times**:
 - `fibonacci(n - 1)`
 - `fibonacci(n - 2)`
- The two returned values are added together.

- This process continues until the base cases $n == 0$ and $n == 1$ are reached.

Error table:

s.no	Error obtained	Rectification
1	Incorrect Fibonacci recursion	Use fibonacci(n-1) + fibonacci(n-2).

Week-2

2a. AIM: Write a python program to perform linear search on the give input.

Code:

```
Go Run Terminal Help AV.SC.U4CSE25008
on even or odd.py practice.py def search_em Launch
1 def linear_search(arr, k):
2     for i in range(len(arr)):
3         if arr[i] == k:
4             return i
5     return -1
6
7 n = int(input("enter the no of elements"))
8 arr = []
9
10 for i in range(n):
11     arr.append(int(input("enter the element:")))
12
13 k = int(input("enter the search element"))
14 print(linear_search(arr, k))
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\Untitled-1.py'
enter the no of elements 2
enter the element:3
enter the element:4
enter the search element 1
-1
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation

Condition 1: Element is Found ($\text{arr}[i] == k$)

- The function `linear_search(arr, k)` receives the array `arr` and the search element `k`.
- The for loop starts from the first element of the array.
- It checks each element one by one using:
 $\text{arr}[i] == k$
- If the current element is equal to `k`, the condition becomes **True**.
- The function immediately returns the **index i**.
- Since the function returns the index, the loop stops immediately.
- The remaining elements are not checked.

Condition 2: Element is Not Found

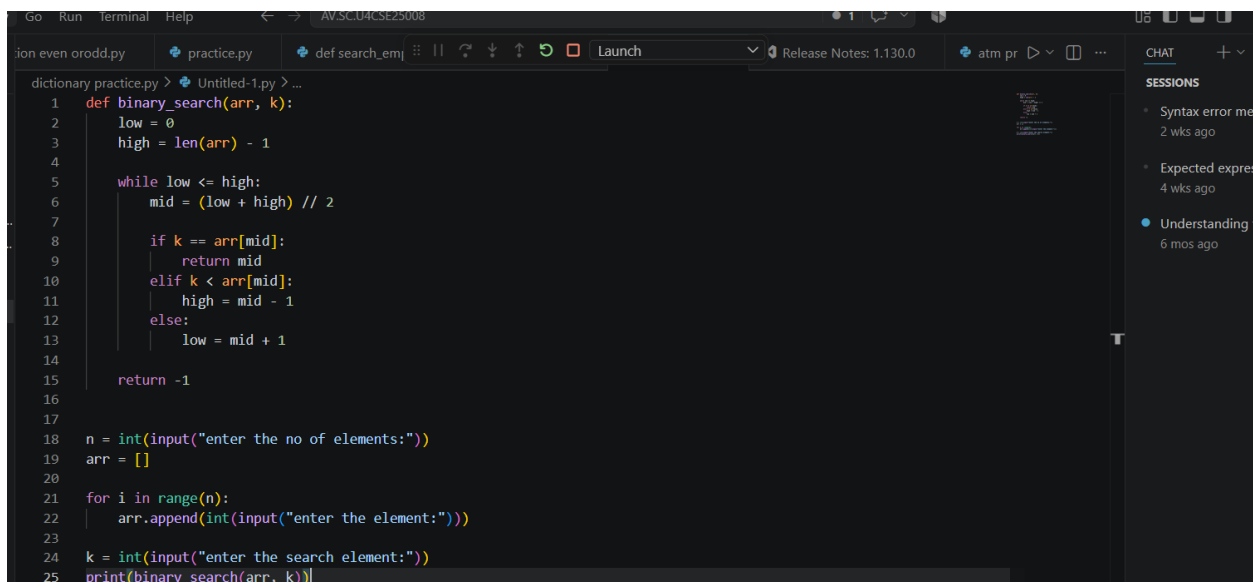
- The loop checks every element in the array.
- If none of the elements is equal to `k`, the if condition is always **False**.
- After the loop finishes, the statement `return -1` is executed.
- `-1` indicates that the search element is **not present** in the array.

Error table:

s.no	Error obtained	Rectification
1	Wrong loop range	Use range(n) to process exactly n elements.

2b. AIM: Write a python program to perform binary search on the given. Verify by taking sorted array as an input.

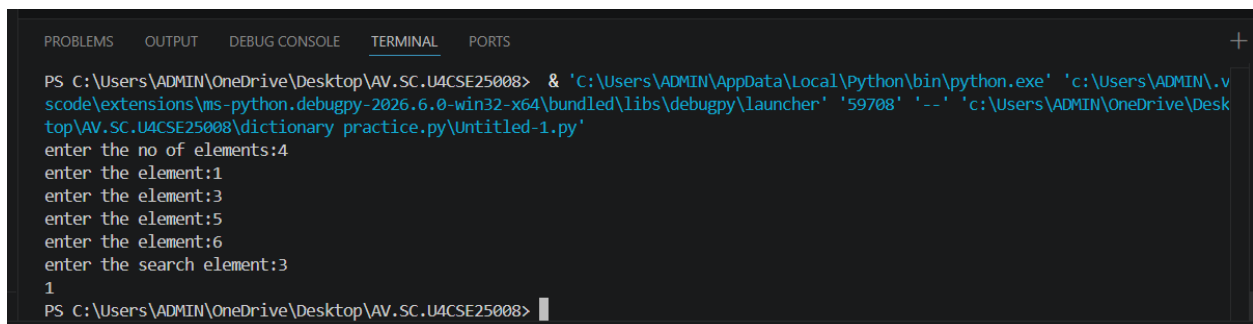
Code:

A screenshot of a Python IDE with a dark theme. The editor shows a file named 'Untitled-1.py' with the following code:

```
1 def binary_search(arr, k):
2     low = 0
3     high = len(arr) - 1
4
5     while low <= high:
6         mid = (low + high) // 2
7
8         if k == arr[mid]:
9             return mid
10        elif k < arr[mid]:
11            high = mid - 1
12        else:
13            low = mid + 1
14
15    return -1
16
17
18 n = int(input("enter the no of elements:"))
19 arr = []
20
21 for i in range(n):
22     arr.append(int(input("enter the element:")))
23
24 k = int(input("enter the search element:"))
25 print(binary_search(arr, k))
```

The right sidebar shows a 'CHAT' panel with a 'SESSIONS' list containing 'Syntax error me' (2 wks ago), 'Expected expres' (4 wks ago), and 'Understanding' (6 mos ago).

Output:

A screenshot of a terminal window with a dark theme. The prompt is 'PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>'. The user enters the command to run the Python script. The output shows the program asking for the number of elements (4), then for each element (1, 3, 5, 6), then for the search element (3), and finally printing '1'.

```
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\Untitled-1.py'
enter the no of elements:4
enter the element:1
enter the element:3
enter the element:5
enter the element:6
enter the search element:3
1
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation

Condition 1: $k == arr[mid]$ — Element Found

- The function `binary_search(arr, k)` receives the array and the search element.

- low is initialized to 0, which represents the first index.
- high is initialized to len(arr)-1, which represents the last index.
- The while loop runs as long as low <= high.
- mid is calculated using:
mid = (low + high) // 2
- The middle element is compared with the search element k.
- If k == arr[mid], the element is found.
- The function returns the **index mid** and stops searching.

Condition 2: k < arr[mid] — Search Left Half

- If k is smaller than the middle element, the condition is **True**.
- This means the search element can only be present in the **left half**.
- Therefore:
high = mid - 1
- The right half is eliminated.
- The process continues with the remaining left half

Condition 3: k > arr[mid] — Search Right Half

- If k is greater than the middle element, the elif condition is **False**.
- The else block is executed.
- This means the search element can only be present in the **right half**.
- Therefore:
low = mid + 1
- The left half is eliminated.
- The process continues with the remaining right half.

Condition 4: low > high — Element Not Found

- The while loop continues until low <= high becomes **False**.
- When low > high, there are no elements left to search.
- This means the search element is not present in the array.
- The function returns **-1**.

Error table:

s.no	Error obtained	Rectification
	Wrong index in binary search	Use high = mid - 1 for the left half and low = mid + 1 for the right half.

```

1 def binary_search(arr, key):
2     low = 0
3     high = len(arr) - 1
4
5     while low <= high:
6         mid = (low + high) // 2
7
8         if arr[mid] == key:
9             return mid
10        elif key < arr[mid]:
11            high = mid - 1
12        else:
13            low = mid + 1
14
15    return -1
16
17 arr = list(map(int, input("Enter the elements: ").split()))
18
19 if arr == sorted(arr):
20     print("Array is already sorted.")
21 else:
22     print("Array is unsorted.")
23     arr.sort()
24     print("Array after sorting:", arr)
25
26 key = int(input("Enter the element to search: "))
27
28 result = binary_search(arr, key)
29
30 if result != -1:
31     print("Element found at index:", result)
32 else:
33     print("Element not found.")
34

```

Output:

```

PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> ^C
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> c::; cd 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008'; & 'C:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55895' '--' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\Untitled-1.py'
Enter the elements: 1 4 2 6 8
Array is unsorted.
Array after sorting: [1, 2, 4, 6, 8]
Enter the element to search: 2
Element found at index: 1
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>

```

Explanation

Condition 1: arr[mid] == key — Element Found

- The function first calculates the middle index using $\text{mid} = (\text{low} + \text{high}) // 2$.
- It compares the middle element $\text{arr}[\text{mid}]$ with the search element key .

- If `arr[mid] == key`, the condition is **True**.
- The element is found.
- The function returns the index `mid`.
- The search stops.

Condition 2: `key < arr[mid]` — Search Left Half

- If `key` is smaller than `arr[mid]`, the condition is **True**.
- This means the required element may be present in the **left half**.
- The right half is ignored.
- `high` is changed to `mid - 1`.
- The search continues in the left half.

Condition 3: `key > arr[mid]` — Search Right Half

- If `key` is greater than `arr[mid]`, the previous conditions are **False**.
- The element may be present in the **right half**.
- The left half is ignored.
- `low` is changed to `mid + 1`.
- The search continues in the right half.

Condition 4: `low > high` — Element Not Found

- The while `low <= high` loop continues until there are elements left to search.
- When `low > high`, the loop stops.
- This means the element is not present in the array.
- The function returns `-1`.
- The program prints **"Element not found."**

Condition 5: Array is Already Sorted

- The program checks whether the given array is already sorted.
- It uses:
`arr == sorted(arr)`
- If the condition is **True**, it prints:
"Array is already sorted."

- The array is directly used for binary search.

Condition 6: Array is Unsorted

- If `arr == sorted(arr)` is **False**, the array is unsorted.
- The program prints:
"Array is unsorted."
- It then uses `arr.sort()` to sort the array.
- The sorted array is displayed.
- Binary search is then performed on the sorted array.

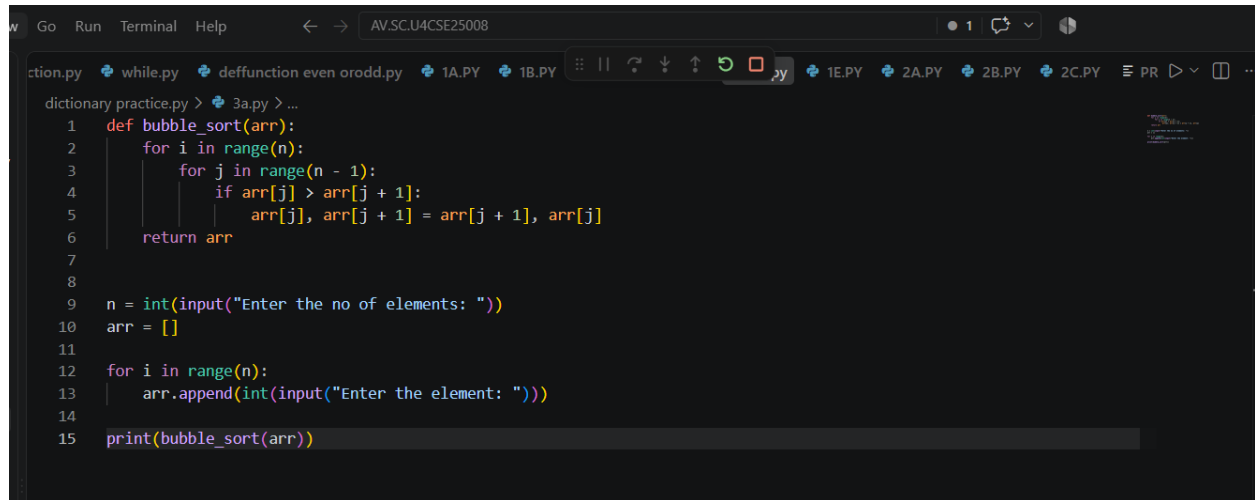
Error table:

s.no	Error obtained	Rectification
1	Binary search on unsorted array	sort the array first using <code>arr.sort()</code> or provide a sorted array.

Week-3

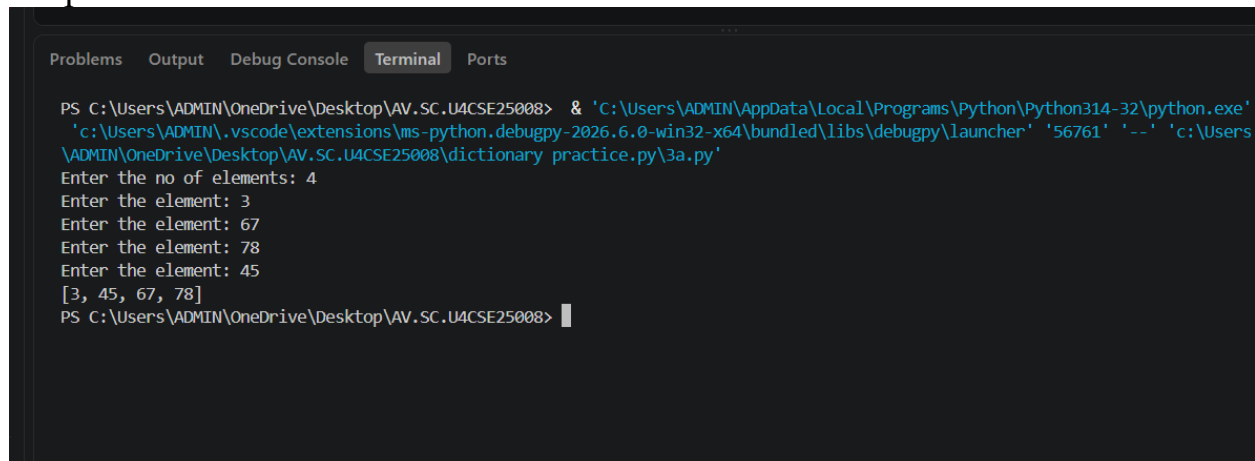
3a.Aim.Write a python program to sort the elements using a bubble sort technique. Take an input elements from the user.

Code:

A screenshot of a Visual Studio Code editor window. The file explorer on the left shows a project named 'dictionary practice.py' with a sub-file '3a.py'. The main editor area displays the code for '3a.py'. The code defines a 'bubble_sort' function that takes an array 'arr' and sorts it using bubble sort. It then prompts the user to enter the number of elements and the elements themselves, and finally prints the sorted array.

```
1 def bubble_sort(arr):
2     for i in range(n):
3         for j in range(n - 1):
4             if arr[j] > arr[j + 1]:
5                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
6     return arr
7
8
9 n = int(input("Enter the no of elements: "))
10 arr = []
11
12 for i in range(n):
13     arr.append(int(input("Enter the element: ")))
14
15 print(bubble_sort(arr))
```

Output:

A screenshot of a terminal window showing the execution of the Python program. The prompt is 'PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>'. The user enters '4' for the number of elements. Then, the user enters '3', '67', '78', and '45' for the elements. The program outputs the sorted array '[3, 45, 67, 78]'.

```
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'c:\Users\ADMIN\AppData\Local\Programs\Python\Python314-32\python.exe'
'c:\Users\ADMIN\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '56761' '--' 'c:\Users
\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\3a.py'
Enter the no of elements: 4
Enter the element: 3
Enter the element: 67
Enter the element: 78
Enter the element: 45
[3, 45, 67, 78]
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation

Condition 1: $arr[j] > arr[j+1]$ — Elements Need to Be Swapped

- The function `bubble_sort(arr)` receives the array as an argument.
- The outer for loop runs from 0 to $n-1$ and controls the number of passes.
- The inner for loop runs from 0 to $n-2$ and compares adjacent elements.
- The condition $arr[j] > arr[j+1]$ checks whether the current element is greater than the next element.
- If the condition is **True**, the two elements are in the wrong order.
- Therefore, the two elements are swapped using:

- `arr[j],arr[j+1]=arr[j+1],arr[j]`
- After the swap, the smaller element moves toward the left and the larger element moves toward the right.

Condition 2: `arr[j] <= arr[j+1]` — No Swap Required

- If `arr[j] > arr[j+1]` is **False**, it means the current element is smaller than or equal to the next element.
- Therefore, the elements are already in the correct ascending order.
- No swapping is performed.
- The inner loop continues to compare the next pair of adjacent elements.

Condition 3: `for i in range(n)` — Repeated Passes

- The outer loop repeats the comparison process for all the elements.
- After each complete pass, the **largest unsorted element moves to the end of the array**.
- The process is repeated until all elements are arranged in ascending order.
- Finally, the function returns the sorted array using:

Error table:

sno	Error obtained	Rectification
1	IndexError: list index out of range	Use correct loop range

3b.Aim. write a python program to sort the elements using a insertion sort technique. Take an input elements from the user.

Code:

```

def insertion_sort(arr):
    for i in range(1, n):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j = j - 1

        arr[j + 1] = key

    return arr

n = int(input("Enter the number of elements: "))
arr = []

for i in range(n):
    arr.append(int(input("Enter the element: ")))

print(insertion_sort(arr))

```

Output:

```
21 print(insertion_sort(arr))

Problems Output Debug Console Terminal Ports

PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Programs\Python\Python314-32\python.exe'
'c:\Users\ADMIN\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '62633' '--' 'c:\Users
\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary_practice.py\3b.py'
Enter the number of elements: 5
Enter the element: 23
Enter the element: 45
Enter the element: 67
Enter the element: 12
Enter the element: 45
[12, 23, 45, 45, 67]
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> |
```

Explanation:

Condition 1: for i in range(1, len(arr)) — Selecting Element

- The loop starts from the **second element**.
- The first element is considered already sorted.
- i represents the element that needs to be placed correctly.

Condition 2: key = arr[i] — Storing Element

- The current element is stored in key.

key = arr[i]

- This element will be compared with the elements before it.

Condition 3: j = i - 1 — Getting Previous Position

- j points to the element just before the current element.

j = i - 1

Condition 4: while j >= 0 and key < arr[j] — Comparing Elements

- The current element is compared with the previous elements.
- If the previous element is greater than key, it is moved one position to the right.

while j >= 0 and key < arr[j]:

arr[j + 1] = arr[j]

j -= 1

- This creates space for the current element.

Condition 5: arr[j + 1] = key — Inserting Element

- The key is placed in its correct position.

arr[j + 1] = key

- The elements before the current position are now sorted.

Condition 6: return arr — Returning Sorted Array

- After all elements are processed, the function returns the sorted array.

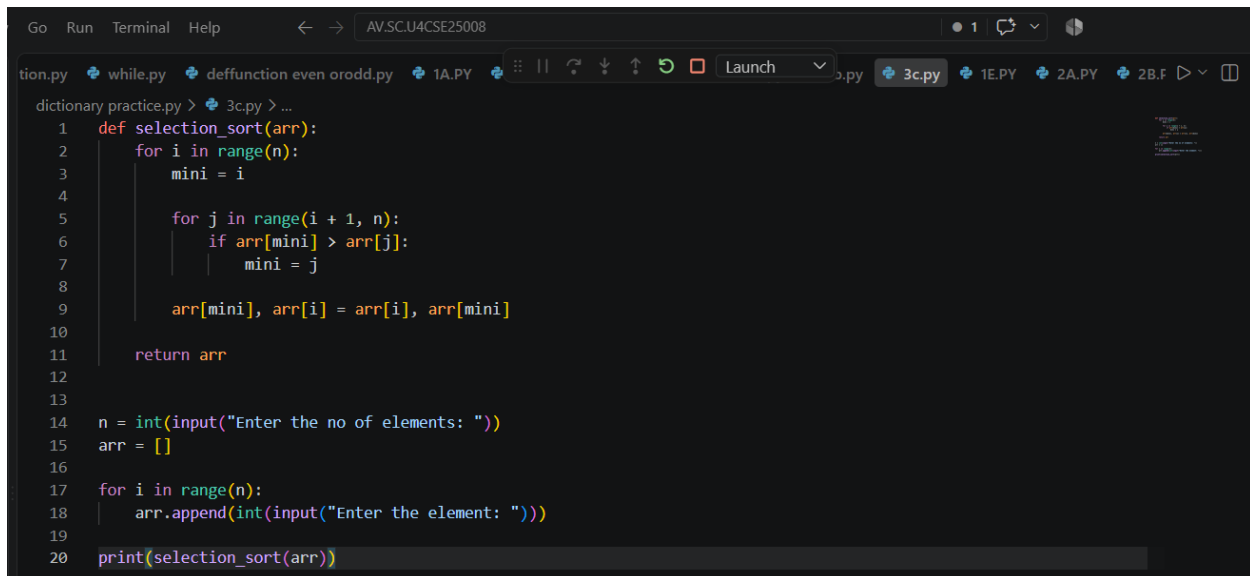
return arr

error table:

sno	Error obtained	Rectification
1	Elements not sorted correctly	Shift elements before insertion

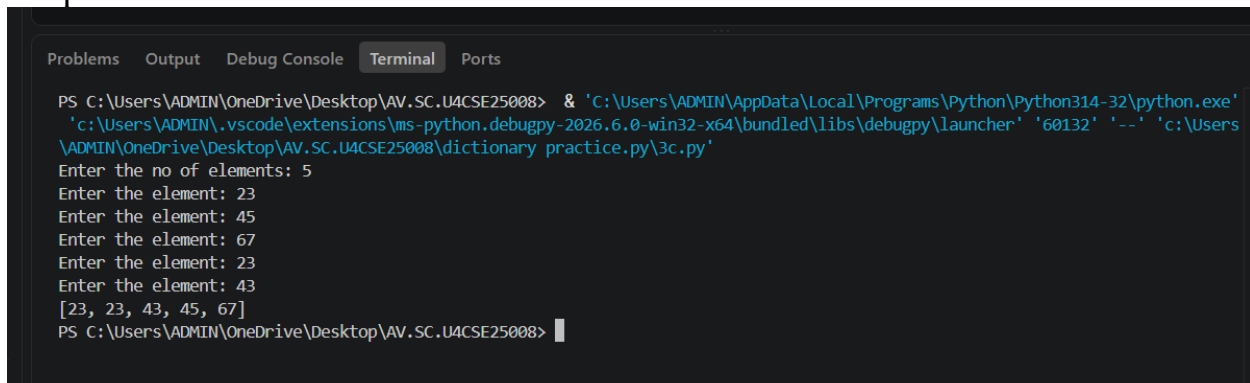
3c.write a python program to sort the elements using selection sort technique . take an input from the user.

Code:

A screenshot of a Visual Studio Code editor window. The top bar shows 'Go Run Terminal Help' and a file path 'AV.SC.U4CSE25008'. The editor has several tabs open: 'tion.py', 'while.py', 'deffunction even orodd.py', '1A.PY', and '3c.py' (which is the active tab). The code in '3c.py' is a Python program for selection sort. It defines a function 'selection_sort(arr)' that takes an array and returns it sorted. The function uses nested loops: an outer loop 'for i in range(n):' and an inner loop 'for j in range(i + 1, n):'. Inside the inner loop, it checks 'if arr[mini] > arr[j]:' and updates 'mini = j'. After the inner loop, it swaps 'arr[mini]' and 'arr[i]'. After the outer loop, it returns 'arr'. Below the function, it takes user input for the number of elements 'n' and the elements themselves, appends them to an array, and prints the result of 'selection_sort(arr)'.

```
1 def selection_sort(arr):
2     for i in range(n):
3         mini = i
4
5         for j in range(i + 1, n):
6             if arr[mini] > arr[j]:
7                 mini = j
8
9         arr[mini], arr[i] = arr[i], arr[mini]
10
11     return arr
12
13
14 n = int(input("Enter the no of elements: "))
15 arr = []
16
17 for i in range(n):
18     arr.append(int(input("Enter the element: ")))
19
20 print(selection_sort(arr))
```

Output:

A screenshot of a terminal window within VS Code. The terminal shows the command prompt 'PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>' followed by the command to run the Python script. The output shows the program prompting for the number of elements (5) and then for each element (23, 45, 67, 23, 43). Finally, it prints the sorted array '[23, 23, 43, 45, 67]'.

```
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Programs\Python\Python314-32\python.exe'
'c:\Users\ADMIN\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '60132' '--' 'c:\Users\
ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\3c.py'
Enter the no of elements: 5
Enter the element: 23
Enter the element: 45
Enter the element: 67
Enter the element: 23
Enter the element: 43
[23, 23, 43, 45, 67]
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation:

Condition 1: for i in range(n) — Selecting Position

- The loop goes through each position in the array.
- i represents the current position where the smallest element will be placed.

Condition 2: mini = i — Assuming Minimum

- Initially, the element at index i is assumed to be the smallest.

Condition 3: for j in range(i+1,n) — Finding Minimum

- The loop checks all elements after position i.
- It searches for the smallest element.

Condition 4: if arr[mini] > arr[j] — Comparing

- Compares the current minimum with arr[j].
- If arr[j] is smaller, mini is updated.

Condition 5: mini = j — Updating Minimum

- Stores the index of the newly found smallest element.

Condition 6: arr[mini], arr[i] = arr[i], arr[mini] — Swapping

- The smallest element is swapped with the element at position i.
- This places the smallest element in its correct position.

Condition 7: return arr — Returning Result

- Returns the completely sorted array.

Error table:

Sno	Error obtained	Rectification
1	Wrong minimum value	Update min_index properly

3d.write a python program to sort the elements using the merge sort technique . take an input from the user.

Code:

```
ew Go Run Terminal Help  AV.SC.U4CSE25008  1 3c.py 3d.py 1E.PY 2A.PY 2B.PY 2C.PY PRINT("HELLO"); Un >
dictionary practice.py > 3d.py > merge_sort
1 def merge_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     mid = len(arr) // 2
5     left = arr[:mid]
6     right = arr[mid:]
7
8     merge_sort(left)
9     merge_sort(right)
10
11     i = j = k = 0
12     while i < len(left) and j < len(right):
13         if left[i] < right[j]:
14             arr[k] = left[i]
15             i += 1
16         else:
17             arr[k] = right[j]
18             j += 1
19             k += 1
20
21     while i < len(left):
22         arr[k] = left[i]
23         i += 1
24         k += 1
25
26     while j < len(right):
27         arr[k] = right[j]
28         j += 1
29         k += 1
30
31 n = int(input("Enter the no of elements: "))
32 arr = []
33 for i in range(n):
34     arr.append(int(input("Enter the element: ")))
35 merge_sort(arr)
36 for element in arr:
37     print(element)
```

Output:

```
Problems Output Debug Console Terminal Ports
Enter the no of elements: 4
Enter the element: 34
Enter the element: 23
Enter the element: 12
Enter the element: 45
12
23
34
45
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation:

Condition 1: if len(arr) <= 1 — Checking the Base Condition

- The function first checks whether the array contains **one or zero elements**.

if len(arr) <= 1:

 return arr

- If the array has one or zero elements, it is already sorted.
- No further division or comparison is required.
- The function returns the array as it is:

return arr

Condition 2: mid = len(arr) // 2 — Finding the Middle

- The array is divided into two parts.
- The middle position is calculated using:

mid = len(arr) // 2

- len(arr) gives the total number of elements.
- // 2 performs integer division and finds the middle index.

Condition 3: left = arr[:mid] — Creating the Left Part

- The elements before the middle position are stored in left.

left = arr[:mid]

Condition 4: right = arr[mid:] — Creating the Right Part

- The elements from the middle position to the end are stored in right.

right = arr[mid:]

Condition 5: merge_sort(left) and merge_sort(right) — Sorting Both Parts

- The left and right parts are recursively sorted.

merge_sort(left)

merge_sort(right)

- The array continues to be divided until each part contains only **one element**.
- A single element is already sorted.
- The smaller sorted parts are then merged together.

Condition 6: i = j = k = 0 — Initializing Indexes

- Three variables are initialized:

i = j = k = 0

- i keeps track of the current element in the **left array**.
- j keeps track of the current element in the **right array**.
- k keeps track of the position in the **original array** where the element will be placed.

Condition 7: while i < len(left) and j < len(right) — Comparing Both Parts

- The program compares elements from the left and right arrays.

while i < len(left) and j < len(right):

- The loop continues as long as both arrays still have elements to compare.
- This allows the program to select the smaller element and place it into the original array.

Condition 8: if left[i] < right[j] — Comparing Elements

- The current elements of the left and right arrays are compared.

if left[i] < right[j]:

- If the left element is smaller than the right element, it is placed into the original array.

arr[k] = left[i]

- Then i is increased:

i += 1

- This moves to the next element of the left array.

Condition 9: else — Taking the Right Element

- If the right element is smaller than or equal to the left element, the right element is placed into the original array.

else:

arr[k] = right[j]

- Then j is increased:

j += 1

- This moves to the next element of the right array.

Condition 10: k += 1 — Moving to the Next Position

- After placing an element into the original array, k is increased.

k += 1

Condition 11: while i < len(left) — Copying Remaining Left Elements

- Sometimes the right array becomes empty before the left array.

while i < len(left):

- Any remaining elements in the left array are copied into the original array.

arr[k] = left[i]

- Then both i and k are increased:

i += 1

k += 1

- The remaining elements are already sorted, so no comparison is needed.

Condition 12: while j < len(right) — Copying Remaining Right Elements

- Sometimes the left array becomes empty before the right array.

while j < len(right):

- Any remaining elements in the right array are copied into the original array.

arr[k] = right[j]

- Then j and k are increased:

j += 1

k += 1

- This completes the merging process.

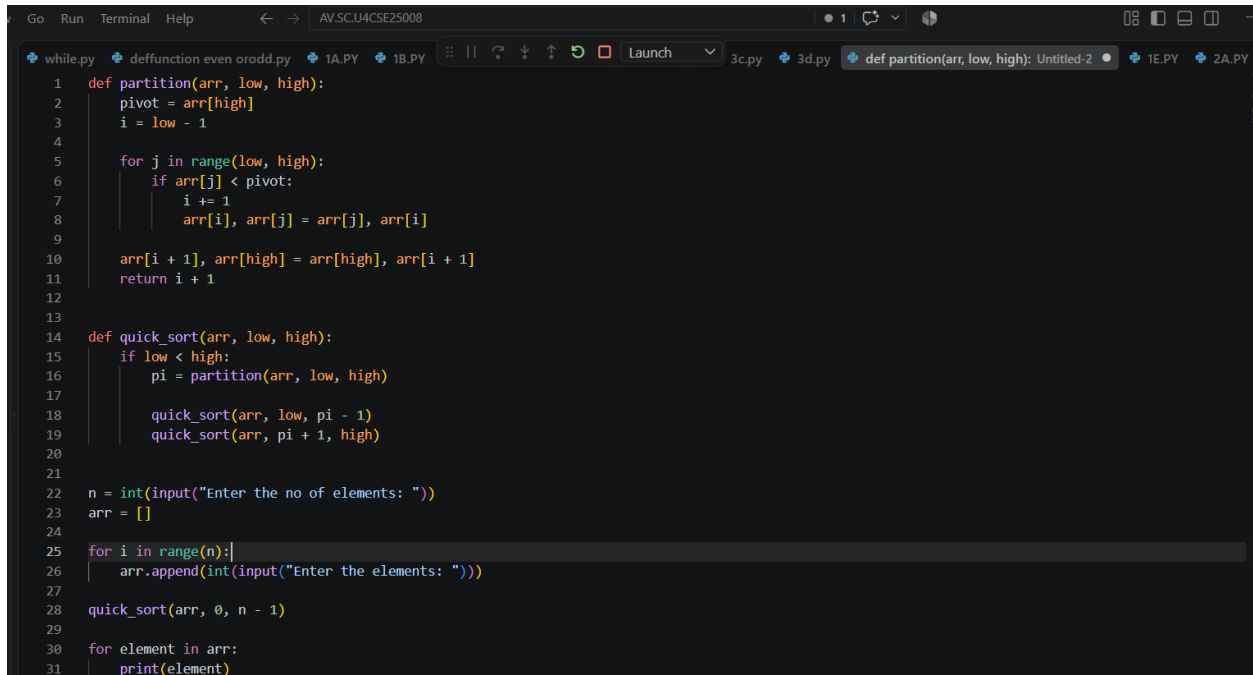
Error table:

sno	Error obtained	Rectification
1	Recursive error	Use if len(arr) <= 1
2	IndexError	Check left/right array indices

Bottom of Form

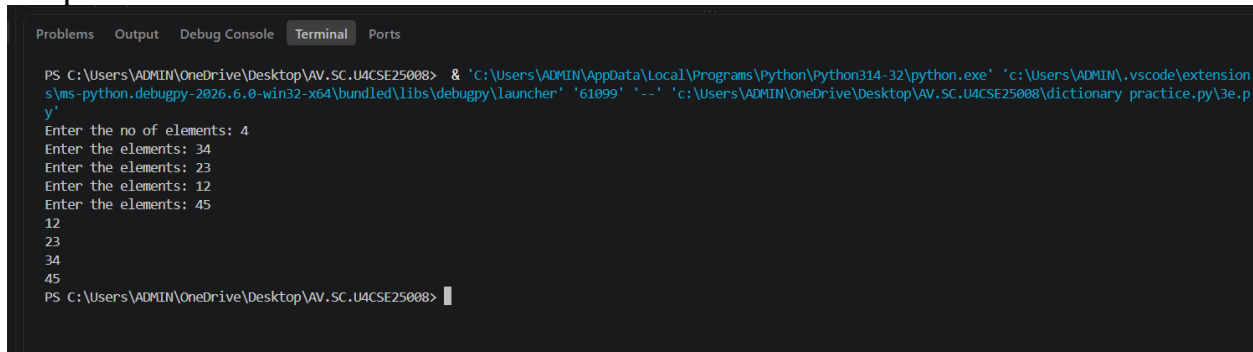
3e. Aim. Write a python program to sort the elements using a quick sort technique . take an input elements from the user

Code:



```
1 def partition(arr, low, high):
2     pivot = arr[high]
3     i = low - 1
4
5     for j in range(low, high):
6         if arr[j] < pivot:
7             i += 1
8             arr[i], arr[j] = arr[j], arr[i]
9
10    arr[i + 1], arr[high] = arr[high], arr[i + 1]
11    return i + 1
12
13
14 def quick_sort(arr, low, high):
15     if low < high:
16         pi = partition(arr, low, high)
17
18         quick_sort(arr, low, pi - 1)
19         quick_sort(arr, pi + 1, high)
20
21
22 n = int(input("Enter the no of elements: "))
23 arr = []
24
25 for i in range(n):
26     arr.append(int(input("Enter the elements: ")))
27
28 quick_sort(arr, 0, n - 1)
29
30 for element in arr:
31     print(element)
```

Output:



```
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Programs\Python\Python314-32\python.exe' 'c:\Users\ADMIN\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '61899' '--' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\3e.py'
Enter the no of elements: 4
Enter the elements: 34
Enter the elements: 23
Enter the elements: 12
Enter the elements: 45
12
23
34
45
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation

Condition 1: if arr[j] < pivot — Element Smaller Than Pivot

- The function partition(arr, low, high) is used to divide the array into two parts.
- pivot = arr[high] selects the **last element** of the array as the pivot.
- i = low - 1 is initialized to keep track of the position where elements smaller than the pivot should be placed.
- The for loop checks each element from index low to high-1.
- The condition arr[j] < pivot checks whether the current element is smaller than the pivot.
- If the condition is **True**, i is increased by 1.
- The current element is then swapped with the element at index i:

`arr[i], arr[j] = arr[j], arr[i]`

- This places all elements smaller than the pivot toward the left side.

Condition 2: `arr[j] >= pivot` — No Swap Required

- If `arr[j] < pivot` is **False**, it means the current element is greater than or equal to the pivot.
- No swapping is performed.
- The loop continues to check the next element.
- These elements will remain on the right side of the pivot after partitioning.

Condition 3: `arr[i+1], arr[high] = arr[high], arr[i+1]` — Placing the Pivot

- After all elements have been checked, the pivot is placed in its correct position.
- The pivot is swapped with the element at index `i+1`:

`arr[i+1], arr[high] = arr[high], arr[i+1]`

- After this operation:
 - Elements smaller than the pivot are on the **left**.
 - The pivot is in its **correct position**.
 - Elements greater than or equal to the pivot are on the **right**.
- The function returns the pivot position:

`return i+1`

Condition 4: if `low < high` — Continue Quick Sort

- The function `quick_sort(arr, low, high)` is used to sort the array.
- The condition `low < high` checks whether the current part of the array contains more than one element.
- If the condition is **True**, the array needs to be sorted.
- The `partition()` function is called:

`pi = partition(arr, low, high)`

- `pi` stores the final position of the pivot.

Condition 5: `quick_sort(arr, low, pi-1)` — Sort Left Part

- After partitioning, the elements on the left side of the pivot are sorted separately.
- This is done using:

`quick_sort(arr, low, pi-1)`

- The pivot itself is not included because it is already in its correct position.

Condition 6: `quick_sort(arr, pi+1, high)` — Sort Right Part

- The elements on the right side of the pivot are sorted separately.
- This is done using:

`quick_sort(arr, pi+1, high)`

- Again, the pivot is not included because it is already correctly positioned.

Input Section

- `n = int(input("enter the no of elements:"))` takes the number of elements from the user.
- `arr = []` creates an empty list to store the elements.
- The for loop runs `n` times and takes each element from the user:
- `for i in range(n):`

`arr.append(int(input("enter the elements:")))`

- Each entered value is converted into an integer and added to the array using `append()`.

Calling the Quick Sort Function

`quick_sort(arr, 0, n-1)`

- This calls the `quick_sort()` function.

- 0 represents the first index of the array.
- n-1 represents the last index of the array.
- The complete array is therefore passed for sorting.

Error table:

sno	Error obtained	Rectification
1	IndexError	Check i, j, and pivot boundaries
2	Wrong swapping conditions	By shifting to the correct spaces